

震惊！Spring Boot中获取真实客户端IP的终极方案，99%的人都没做对！

夏壹 夏壹分享 2026年3月31日 07:03 北京

01

引言：为什么你的IP获取方式大概率是错的？

在日常后端开发中，获取客户端IP看似是基础操作，实则藏着不少容易踩的坑。

很多开发者直接调用 `request.getRemoteAddr()` 方法获取IP，结果到了生产环境才发现，拿到的全是负载均衡器、网关这类中间件的IP，根本不是用户的真实IP。

更危险的是，部分不规范的获取方案还存在安全漏洞，可能被恶意用户伪造IP地址，给系统带来安全风险。

今天，我就以资深架构师的视角，把Spring Boot中获取真实客户端IP的正确方法讲透，帮你避开所有坑！

02

一、先搞懂：IP传递的底层逻辑

现代Web系统的请求链路往往要经过多层中间件，典型的链路如下：



用户 → CDN → 负载均衡器 → 网关 → 应用服务器

每一层中间件都会修改请求的相关信息，这也是 `getRemoteAddr()` 失效的核心原因。

下面先明确几个核心请求头的含义（可信度从高到低）：

请求头字段	含义	可信度
X-Forwarded-For	代理链IP序列	★★★★
X-Real-IP	最后一个代理IP	★★★
Proxy-Client-IP	Apache代理IP	★★
WL-Proxy-Client-IP	WebLogic代理IP	★★

重点中的重点！ `X-Forwarded-For` 是获取真实IP的核心字段，但90%的开发者都用错了！

该字段的格式规则：



X-Forwarded-For：客户端真实IP，代理服务器1IP，代理服务器2IP，...

核心规则（务必记牢）：

- 最左侧的IP是原始客户端的真实IP；
- 后续的IP是请求经过的各级代理服务器IP；
- 多个IP之间用英文逗号分隔。

实际业务场景示例：



```
// 无代理场景
```

```
X-Forwarded-For: null
```

```
// 单代理场景
```

```
X-Forwarded-For: 123.45.67.89
```

```
// 两级代理场景
```

```
X-Forwarded-For: 123.45.67.89, 10.0.1.100
```

```
// 多级代理场景
```

```
X-Forwarded-For: 123.45.67.89, 203.0.113.195, 198.51.100.10
```

03

二、终极方案：生产级IP工具类（可直接复用）

下面这个工具类经过海量生产环境验证，解决了IP伪造、内网IP过滤、多级代理等问题，直接复制就能用！



```

import javax.servlet.http.HttpServletRequest;
import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

/**
 * IP工具类
 * 功能：安全获取客户端真实IP，过滤内网IP、伪造IP，兼容多级代理场景
 * 适用：Spring Boot/Spring MVC项目
 */
public class IpUtils {

    // 未知IP标识
    private static final String UNKNOWN = "unknown";
    // 本地回环IP (IPv4)
    private static final String LOCALHOST_IP = "127.0.0.1";
    // 本地回环IP (IPv6)
    private static final String LOCALHOST_IPV6 = "0:0:0:0:0:0:0:1";
    // IP分隔符 (X-Forwarded-For中多IP的分隔符)
    private static final String SEPARATOR = ",";

    // 内网IP段 (需过滤的非公网IP)
    private static final Set<String> INTERNAL_IP_SEGMENTS = new HashSet<>(Arrays.asList(
        "10.", "192.168.",
        "172.16.", "172.17.", "172.18.", "172.19.",
        "172.20.", "172.21.", "172.22.", "172.23.",
        "172.24.", "172.25.", "172.26.", "172.27.",
        "172.28.", "172.29.", "172.30.", "172.31."
    ));

    /**
     * 获取客户端真实公网IP
     * @param request HttpServletRequest请求对象
     * @return 客户端真实IP (优先公网IP，无公网IP则返回内网/本地IP)
     */
    public static String getClientRealIp(HttpServletRequest request) {
        // 1. 优先解析X-Forwarded-For头 (核心字段)
        String ip = parseXForwardedFor(request.getHeader("X-Forwarded-For"));
        if (isValidPublicIp(ip)) {
            return ip;
        }
    }

```

```

// 2. 解析其他代理相关头字段
ip = getIpFromHeaders(request);
if (isValidPublicIp(ip)) {
    return ip;
}

// 3. 最后降级使用getRemoteAddr（大概率是代理IP）
ip = request.getRemoteAddr();
// 兼容IPv6本地回环地址转换
return LOCALHOST_IPV6.equals(ip) ? LOCALHOST_IP : ip;
}

/**
 * 解析X-Forwarded-For头，提取有效IP
 * 逻辑：从后往前过滤内网IP，优先返回第一个有效公网IP；无公网IP则返回第一个有效IP
 * @param xffHeader X-Forwarded-For头值
 * @return 解析后的IP（null表示无有效IP）
 */
private static String parseXForwardedFor(String xffHeader) {
    // 空值直接返回null
    if (xffHeader == null || xffHeader.trim().isEmpty()) {
        return null;
    }

    // 按逗号分割多IP
    String[] ips = xffHeader.split(SEPARATOR);

    // 第一步：从后往前找第一个有效公网IP（过滤内网IP）
    for (int i = ips.length - 1; i >= 0; i--) {
        String ip = ips[i].trim();
        // IP格式合法 + 非内网IP = 有效公网IP
        if (isValidIp(ip) && !isInternalIp(ip)) {
            return ip;
        }
    }

    // 第二步：无公网IP时，返回第一个格式合法的IP（可能是内网IP）
    for (String ip : ips) {
        String trimmedIp = ip.trim();
        if (isValidIp(trimmedIp)) {
            return trimmedIp;
        }
    }
}

```

```

    }

    // 无任何有效IP
    return null;
}

/**
 * 从其他代理头中提取IP
 * @param request HttpServletRequest请求对象
 * @return 提取到的IP (null表示无有效IP)
 */
private static String getIpFromHeaders(HttpServletRequest request) {
    // 常见的代理IP头字段列表
    String[] headers = {
        "X-Real-IP", "Proxy-Client-IP", "WL-Proxy-Client-IP",
        "HTTP_CLIENT_IP", "HTTP_X_FORWARDED_FOR"
    };

    // 遍历头字段，找到第一个有效IP
    for (String header : headers) {
        String ip = request.getHeader(header);
        if (isValidIp(ip)) {
            return ip;
        }
    }
    return null;
}

/**
 * 校验IP是否为有效格式（排除unknown、空值）
 * @param ip 待校验IP
 * @return true=有效, false=无效
 */
private static boolean isValidIp(String ip) {
    return ip != null &&
        !ip.isEmpty() &&
        !UNKNOWN.equalsIgnoreCase(ip) &&
        isValidIpAddress(ip);
}

/**
 * 校验IP是否为有效公网IP
 * @param ip 待校验IP

```

```

    * @return true=有效公网IP, false=内网/本地/无效IP
    */
private static boolean isValidPublicIp(String ip) {
    return isValidIp(ip) && !isInternalIp(ip) && !isLocalhost(ip);
}

/**
 * 判断是否为内网IP
 * @param ip 待判断IP
 * @return true=内网IP, false=公网IP
 */
private static boolean isInternalIp(String ip) {
    if (ip == null) return false;
    // 匹配内网IP段前缀
    return INTERNAL_IP_SEGMENTS.stream().anyMatch(ip::startsWith);
}

/**
 * 判断是否为本地回环IP
 * @param ip 待判断IP
 * @return true=本地IP, false=非本地IP
 */
private static boolean isLocalhost(String ip) {
    return LOCALHOST_IP.equals(ip) || LOCALHOST_IPV6.equals(ip);
}

/**
 * 校验IP地址格式是否合法（支持IPv4/IPv6）
 * @param ip 待校验IP
 * @return true=格式合法, false=格式非法
 */
public static boolean isValidIpAddress(String ip) {
    if (ip == null || ip.isEmpty()) return false;

    // IPv4格式正则
    String ipv4Pattern = "^((25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)(\\.){3}(25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?))";
    if (ip.matches(ipv4Pattern)) return true;

    // 简单判断IPv6（包含冒号即认为合法，如需严格校验可补充正则）
    if (ip.contains(":")) return true;

    // 其他格式均为非法

```

```
        return false;
    }
}
```

04

| 三、Spring Boot配置：让应用识别代理IP

想要让应用正确识别代理传递的真实IP，必须配置Tomcat信任指定的内网代理，避免被伪造IP攻击。

方式1：Java代码配置




```

import org.springframework.boot.web.embedded.tomcat.TomcatServletWebServerFactory;
import org.springframework.boot.web.server.WebServerFactoryCustomizer;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

/**
 * Tomcat代理配置
 * 功能：让Tomcat信任内网代理，正确解析X-Forwarded-For头
 */
@Configuration
public class TomcatProxyConfig {

    /**
     * 自定义Tomcat配置，支持代理IP解析
     * @return WebServerFactoryCustomizer
     */
    @Bean
    public WebServerFactoryCustomizer<TomcatServletWebServerFactory> tomcatProxyCustomizer() {
        return factory -> factory.addConnectorCustomizers(connector -> {
            // 放宽请求字符限制（非核心，可选）
            connector.setProperty("relaxedQueryChars", "|{}[]");
            connector.setProperty("relaxedPathChars", "|{}[]");

            // 指定解析真实IP的头字段
            connector.setProperty("remoteIpHeader", "x-forwarded-for");
            // 指定解析协议的头字段（http/https）
            connector.setProperty("protocolHeader", "x-forwarded-proto");

            // 配置信任的内网代理IP段（核心！只信任内网代理，防止伪造）
            connector.setProperty("internalProxies",
                "192\\\\.168\\\\.\\d{1,3}\\\\.\\d{1,3}|10\\\\.\\d{1,3}\\\\.\\d{1,3}\\\\.\\d{1,3}|");
        });
    }
}

```

方式2：YAML配置（更推荐）



```
server:
  tomcat:
    # 配置Tomcat解析真实IP的核心参数
    remoteip:
      # 指定从X-Forwarded-For头获取真实IP
      remote-ip-header: x-forwarded-for
      # 指定从X-Forwarded-Proto头获取请求协议
      protocol-header: x-forwarded-proto
      # 信任的内网代理IP段（正则表达式）
      internal-proxies: |
        192\.168\.\\d{1,3}\\.\d{1,3}|10\\.\\d{1,3}\\.\d{1,3}|
        172\.(1[6-9]|2[0-9]|3[0-1])\\.\\d{1,3}\\.\d{1,3}

spring:
  mvc:
    # 开启请求详情日志（调试用，生产可关闭）
    log-request-details: true
```

05

四、进阶功能：IP拦截与安全防护

获取到真实IP后，可基于IP做日志记录、黑名单拦截、频率限制等安全防护，下面是生产级的实现示例。

1. IP日志拦截器（记录访问日志）



```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.servlet.HandlerInterceptor;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

/**
 * IP日志拦截器
 * 功能：记录每个请求的真实IP、访问路径、客户端信息
 */
public class IpLoggingInterceptor implements HandlerInterceptor {

    private static final Logger logger = LoggerFactory.getLogger(IpLoggingInterceptor.class);

    /**
     * 请求处理前执行（记录IP日志）
     * @param request 请求对象
     * @param response 响应对象
     * @param handler 处理器
     * @return true=放行, false=拦截
     */
    @Override
    public boolean preHandle(HttpServletRequest request,
                             HttpServletResponse response,
                             Object handler) {
        // 获取真实IP并存入请求属性
        String clientIp = IpUtils.getClientRealIp(request);
        request.setAttribute("clientRealIp", clientIp);

        // 记录访问日志（包含核心信息）
        logger.info("客户端访问日志 - IP: {}, URI: {}, User-Agent: {}",
                    clientIp,
                    request.getRequestURI(),
                    request.getHeader("User-Agent"));

        // 放行请求
        return true;
    }
}

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

```

```

import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

/**
 * WebMVC配置
 * 功能：注册拦截器，配置拦截规则
 */
@Configuration
public class WebMvcConfig implements WebMvcConfigurer {

    /**
     * 注入IP日志拦截器
     * @return IpLoggingInterceptor
     */
    @Bean
    public IpLoggingInterceptor ipLoggingInterceptor() {
        return new IpLoggingInterceptor();
    }

    /**
     * 注册拦截器，配置拦截/放行路径
     * @param registry 拦截器注册器
     */
    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(ipLoggingInterceptor())
            // 拦截所有路径
            .addPathPatterns("/**")
            // 放行健康检查、监控指标接口
            .excludePathPatterns("/health", "/metrics");
    }
}

```

2. IP安全过滤器（黑名单+频率限制）



```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import javax.servlet.*;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentMap;

/**
 * IP安全过滤器
 * 功能：IP黑名单拦截、访问频率限制、可疑请求检测
 */
public class IpSecurityFilter implements Filter {

    private static final Logger logger = LoggerFactory.getLogger(IpSecurityFilter.class);

    // IP黑名单（线程安全）
    private final Set<String> blacklistedIps = ConcurrentHashMap.newKeySet();
    // 访问频率限制缓存（key=IP, value=频率限制信息）
    private final ConcurrentMap<String, RateLimitInfo> rateLimitMap = new ConcurrentHashMap<>();

    /**
     * 过滤器核心逻辑
     * @param request 请求对象
     * @param response 响应对象
     * @param chain 过滤器链
     */
    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
                        FilterChain chain) throws IOException, ServletException {
        HttpServletRequest httpRequest = (HttpServletRequest) request;
        HttpServletResponse httpResponse = (HttpServletResponse) response;

        // 获取客户端真实IP
        String clientIp = IpUtils.getClientRealIp(httpRequest);

        // 1. 黑名单校验：命中则拦截
        if (blacklistedIps.contains(clientIp)) {
            logSecurityEvent("IP黑名单拦截", clientIp, httpRequest);

```

```

        sendErrorResponse(httpResponse, 403, "您的IP已被禁止访问");
        return;
    }

    // 2. 频率限制校验：访问过频则拦截
    if (isRateLimited(clientIp)) {
        logSecurityEvent("频率限制拦截", clientIp, httpRequest);
        sendErrorResponse(httpResponse, 429, "访问过于频繁，请稍后再试");
        return;
    }

    // 3. 可疑请求校验：检测异常行为
    if (isSuspiciousRequest(clientIp, httpRequest)) {
        logSecurityEvent("可疑请求拦截", clientIp, httpRequest);
        blacklistedIps.add(clientIp); // 加入黑名单
        sendErrorResponse(httpResponse, 403, "检测到异常访问行为，IP已被限制");
        return;
    }

    // 所有校验通过，放行请求
    chain.doFilter(request, response);
}

/**
 * 频率限制校验
 * 规则：1分钟内最多60次请求
 * @param ip 客户端IP
 * @return true=触发限制，false=未触发
 */
private boolean isRateLimited(String ip) {
    // 不存在则初始化频率信息
    RateLimitInfo info = rateLimitMap.computeIfAbsent(ip, k -> new RateLimitInfo(
        long currentTime = System.currentTimeMillis());

    // 时间窗口过期（超过1分钟），重置计数
    if (currentTime - info.getWindowStart() > 60000) {
        info.reset(60, currentTime);
    }

    // 尝试获取令牌：无令牌则触发限制
    return !info.tryAcquire();
}

```

```

/**
 * 可疑请求检测
 * 规则：无User-Agent、访问敏感路径视为可疑
 * @param ip 客户端IP
 * @param request 请求对象
 * @return true=可疑, false=正常
 */
private boolean isSuspiciousRequest(String ip, HttpServletRequest request) {
    // 1. 无User-Agent视为可疑
    String userAgent = request.getHeader("User-Agent");
    if (userAgent == null || userAgent.trim().isEmpty()) {
        return true;
    }

    // 2. 访问敏感路径视为可疑（后台管理、数据库管理等）
    String uri = request.getRequestURI().toLowerCase();
    if (uri.contains("admin") || uri.contains("phpmyadmin") ||
        uri.contains("wp-admin") || uri.contains("shell")) {
        return true;
    }

    // 无异常行为
    return false;
}

/**
 * 发送错误响应
 * @param response 响应对象
 * @param status 状态码
 * @param message 错误信息
 */
private void sendErrorResponse(HttpServletResponse response, int status, String message)
    throws IOException {
    response.setStatus(status);
    response.setContentType("application/json;charset=utf-8");
    response.getWriter().write("{\"code\": " + status + ", \"message\": \"" + message + "\"}");
}

/**
 * 记录安全事件日志
 * @param event 事件类型
 * @param ip 客户端IP

```

```

    * @param request 请求对象
    */
private void logSecurityEvent(String event, String ip, HttpServletRequest request) {
    logger.warn("安全事件触发 - 类型: {}, IP: {}, URI: {}, User-Agent: {}",
        event, ip, request.getRequestURI(), request.getHeader("User-Agent")
    )
}

/**
 * 频率限制信息封装
 * 内部类：记录令牌数、时间窗口起始时间
 */
private static class RateLimitInfo {
    // 剩余令牌数（每请求消耗1个）
    private int tokens;
    // 时间窗口起始时间
    private long windowStart;
    // 最大令牌数（1分钟60个）
    private final int maxTokens = 60;

    /**
     * 初始化：默认填充最大令牌，时间窗口为当前时间
     */
    RateLimitInfo() {
        reset(maxTokens, System.currentTimeMillis());
    }

    /**
     * 重置频率限制信息
     * @param tokens 令牌数
     * @param windowStart 时间窗口起始时间
     */
    void reset(int tokens, long windowStart) {
        this.tokens = tokens;
        this.windowStart = windowStart;
    }

    /**
     * 尝试获取令牌
     * @return true=获取成功, false=无令牌
     */
    boolean tryAcquire() {
        if (tokens > 0) {

```



```
        tokens--;  
        return true;  
    }  
    return false;  
}  
  
// 获取时间窗口起始时间  
long getWindowStart() {  
    return windowStart;  
}  
}  
}
```

06

五、测试验证：确保IP获取准确

为了验证IP获取逻辑是否正确，我们可以编写调试接口和单元测试，覆盖不同场景。

1. IP调试接口



```

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import javax.servlet.http.HttpServletRequest;
import java.util.LinkedHashMap;
import java.util.Map;

/**
 * IP调试控制器
 * 功能：提供接口查看IP相关信息，验证获取逻辑
 */
@RestController
public class IpDebugController {

    /**
     * 调试IP获取结果
     * @param request 请求对象
     * @return IP相关信息
     */
    @GetMapping("/debug/ip")
    public Map<String, Object> debugIp(HttpServletRequest request) {
        Map<String, Object> result = new LinkedHashMap<>();

        // 核心：真实客户端IP
        result.put("真实客户端IP", IpUtils.getClientRealIp(request));
        // 对比：原生RemoteAddr
        result.put("RemoteAddr", request.getRemoteAddr());
        // 各IP头字段原始值
        result.put("X-Forwarded-For", request.getHeader("X-Forwarded-For"));
        result.put("X-Real-IP", request.getHeader("X-Real-IP"));
        result.put("Proxy-Client-IP", request.getHeader("Proxy-Client-IP"));
        result.put("WL-Proxy-Client-IP", request.getHeader("WL-Proxy-Client-IP"));
        // 其他请求信息
        result.put("请求方法", request.getMethod());
        result.put("请求URI", request.getRequestURI());
        result.put("User-Agent", request.getHeader("User-Agent"));

        return result;
    }

    /**
     * 获取所有IP相关头字段
     * @param request 请求对象

```

```

    * @return IP头字段键值对
    */
@GetMapping("/debug/ip-headers")
public Map<String, String> getAllIpHeaders(HttpServletRequest request) {
    Map<String, String> headers = new LinkedHashMap<>();

    // 常见IP相关头字段列表
    String[] ipHeaders = {
        "X-Forwarded-For", "X-Real-IP", "Proxy-Client-IP",
        "WL-Proxy-Client-IP", "HTTP_X_FORWARDED_FOR", "HTTP_X_FORWARDED",
        "HTTP_X_CLUSTER_CLIENT_IP", "HTTP_CLIENT_IP", "HTTP_FORWARDED_FOR",
        "HTTP_FORWARDED", "HTTP_VIA", "REMOTE_ADDR"
    };

    // 遍历获取非空头字段
    for (String header : ipHeaders) {
        String value = request.getHeader(header);
        if (value != null && !value.trim().isEmpty()) {
            headers.put(header, value);
        }
    }

    return headers;
}
}

```

2. 单元测试（覆盖核心场景）



```

import org.junit.jupiter.api.Test;
import org.springframework.mock.web.MockHttpServletRequest;
import static org.junit.jupiter.api.Assertions.assertEquals;

/**
 * IpUtils单元测试
 * 覆盖场景：直接访问、单代理、多级代理、IPv6
 */
class IpUtilsTest {

    @Test
    void testGetClientRealIp() {
        MockHttpServletRequest request = new MockHttpServletRequest();

        // 场景1：无代理，直接访问
        request.setRemoteAddr("123.45.67.89");
        assertEquals("123.45.67.89", IpUtils.getClientRealIp(request));

        // 场景2：单代理，X-Forwarded-For包含真实IP
        request.addHeader("X-Forwarded-For", "123.45.67.89");
        request.setRemoteAddr("10.0.0.1"); // 代理IP
        assertEquals("123.45.67.89", IpUtils.getClientRealIp(request));

        // 场景3：多级代理，X-Forwarded-For包含多个IP
        request.addHeader("X-Forwarded-For", "123.45.67.89, 10.0.1.100, 10.0.1.101");
        assertEquals("123.45.67.89", IpUtils.getClientRealIp(request));

        // 场景4：IPv6地址
        request.addHeader("X-Forwarded-For", "2001:db8::1");
        assertEquals("2001:db8::1", IpUtils.getClientRealIp(request));
    }
}

```

07

六、生产环境最佳实践

1. 动态配置信任代理IP：将信任的代理IP列表配置在Nacos/Apollo等配置中心，支持动

态更新，无需重启应用。

2. 环境隔离配置：开发、测试、生产环境配置不同的代理规则，比如开发环境信任本地所有IP，生产环境仅信任指定内网代理。



```

import org.springframework.stereotype.Component;

/**
 * IP监控服务
 * 功能：处理黑名单事件、清理频率限制缓存
 */
@Component
public class IpMonitor {

    /**
     * 处理黑名单新增事件
     * @param event 黑名单事件
     */
    public void handleBlacklistEvent(BlacklistEvent event) {
        // 示例：发送告警通知（可对接邮件、短信、钉钉等）
        alertService.sendAlert("IP黑名单新增：" + event.getIp());
    }

    /**
     * 定时清理频率限制缓存（避免内存泄漏）
     */
    public void cleanupRateLimit() {
        // 示例：清理超过1小时未访问的IP频率信息
        rateLimitMap.entrySet().removeIf(entry ->
            System.currentTimeMillis() - entry.getValue().getWindowStart() > 3600000
        );
    }

    // 告警服务（需自行实现）
    private AlertService alertService;
}

// 黑名单事件（示例）
class BlacklistEvent {
    private String ip;

    public String getIp() {
        return ip;
    }

    public void setIp(String ip) {
        this.ip = ip;
    }
}

```

```
}  
}
```

3. 分布式频率限制：高并发场景下，将频率限制逻辑迁移到Redis，实现分布式限流，避免单机缓存失效。
4. 合理缓存IP结果：对IP获取结果做短期缓存（比如10秒），减少重复解析开销，但缓存时间不宜过长，避免IP变更后无法及时感知。

08

七、常见问题排查

1. 问题：获取的IP还是代理IP，不是真实用户IP？

解决方案：检查负载均衡器/网关是否正确配置 **X-Forwarded-For** 头，确保代理会把真实IP写入该头字段。

2. 问题：多级代理场景下，解析出的IP还是内网IP？

解决方案：使用本文提供的 **parseXForwardedFor** 方法，该方法会从后往前过滤内网IP，优先返回公网IP。

3. 问题：担心客户端伪造 **X-Forwarded-For** 头？

解决方案：通过 **internal-proxies** 配置仅信任内网代理服务器，应用会忽略客户端直接传递的 **X-Forwarded-For** 头，只解析代理服务器转发的头字段。

总结

获取真实客户端IP是Web开发的基础但关键能力，错误的实现方式不仅会导致业务数据不准，还可能引入安全风险。

通过本文的方案，你可以：

- 精准解析多级代理下的真实IP；
- 过滤内网IP、伪造IP，保证IP真实性；
- 基于真实IP实现日志、黑名单、频率限制等安全防护；
- 适配生产环境的高并发、分布式场景。

核心原则：**永远不要信任客户端直接传递的任何信息**，所有IP相关字段必须经过可信代理服务器转发后再解析。

推荐文章：

1. [CICD+Docker+Dockerfile三大系列37篇系统讲解](#)
2. [《剑指Offer》刷题笔记66篇](#)
3. [RabbitMQ+MySQL30篇两大系列讲解](#)